1.7 Direct Memory Access

- Three techniques are possible for I/O operations
  - Needs CPU interfering
    - programmed I/O
    - interrupt-driven I/O
  - direct memory access (DMA)

Programmed I/O

- The processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of more data.
- The processor, while waiting, must repeatedly interrogate the status of the I/O module.
- I/O module doesn’t interrupt the processor.
- As a result, the performance level of the entire system is severely degraded.

Interrupt-driven I/O

- More efficient than simple programmed I/O.
- Still requires the active intervention of the processor to transfer data between memory and an I/O module.
- Interrupts the processor.
- Any data transfer must traverse a path through the processor.

Direct Memory Access (DMA)

- Is mainly used to move large volumes of data.
- When the processor wishes to read or write a block of data, it issues a command to the DMA module by sending the following information:
  - Whether a read or write is requested
  - The address of the I/O device involved
  - The starting location in memory to read data from or write data to
  - The number of words to be read or written
- The processor is involved only at the beginning and end of the transfer.

Chapter 2

Sections 2.1, 2.2, 2.3, and 2.4

Section 2.1: Operating System Objectives and Functions

- An OS is a program that controls the execution of application programs,
- Acts as an interface between applications and the computer hardware.
- OS has three main objectives:
  - **Convenience:** An OS makes a computer more easy to use.
  - **Efficiency:** An OS allows the computer system resources to be used in an efficient manner.
  - **Ability to evolve:** permit the effective development, testing, and introduction of new system functions

The Operating System as a User/Computer Interface

- The user of applications generally is not concerned with the details of computer hardware.
- The end user views a computer system in terms of a set of applications.

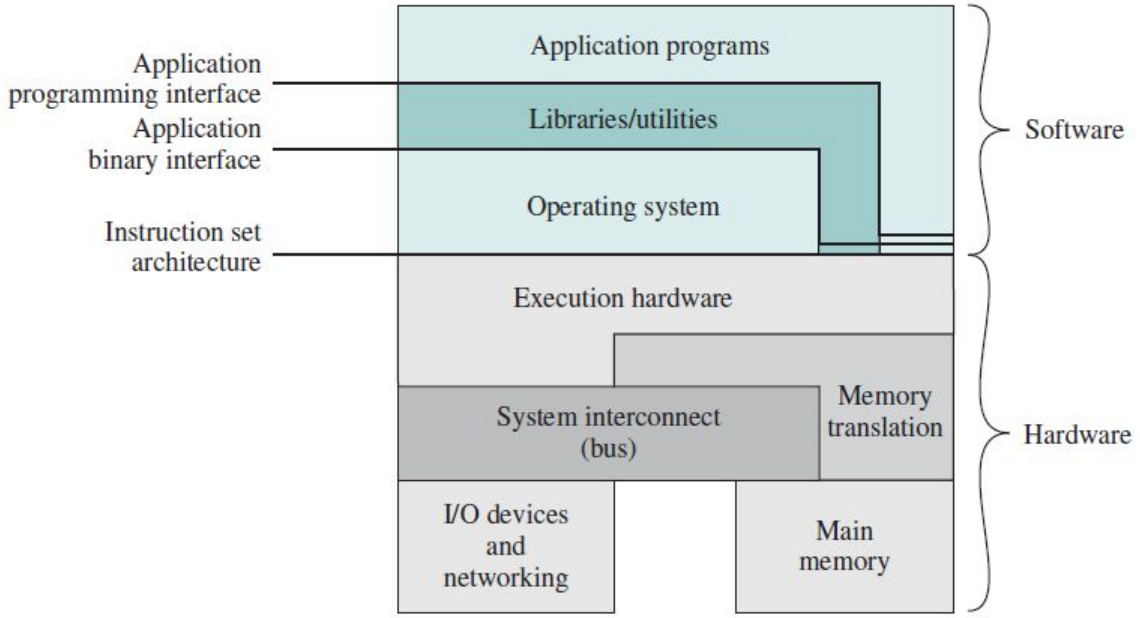


Figure 2.1 Computer Hardware and Software Structure

OS typically provides services in the following areas:

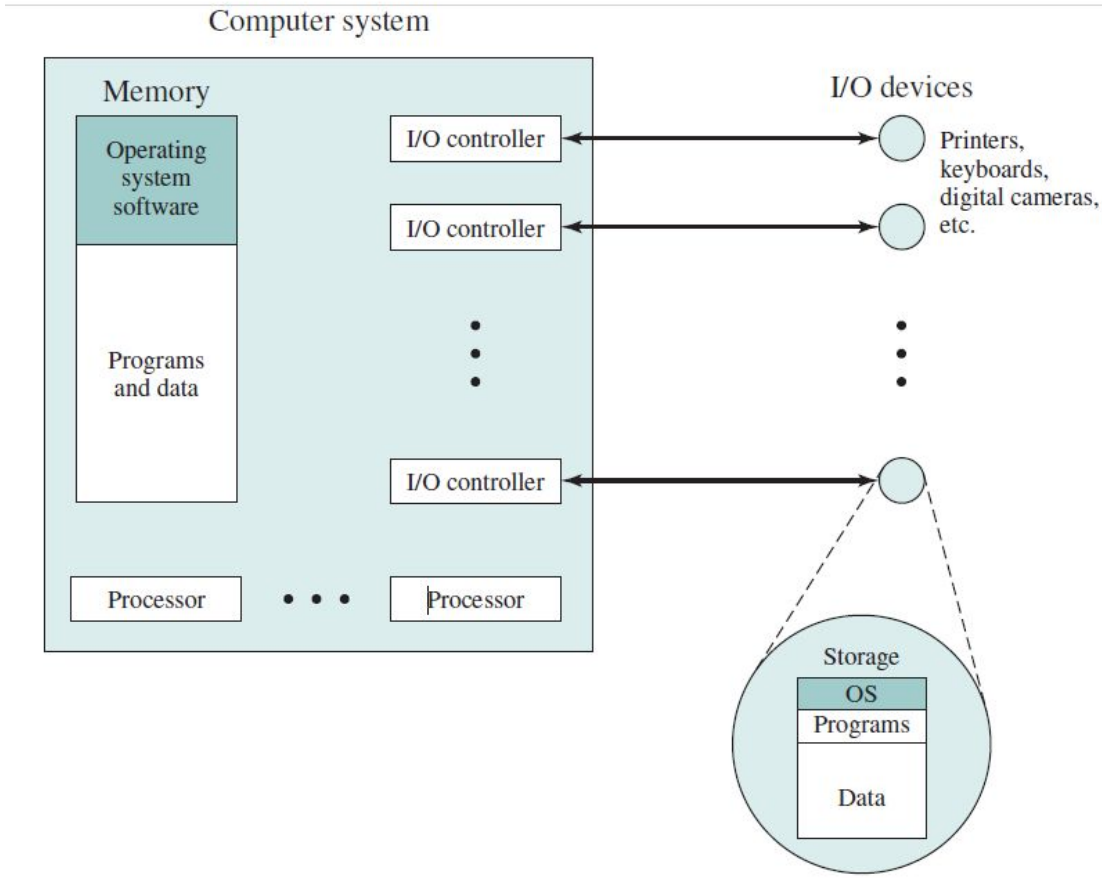
- **Program development:** OS provide editors and debuggers
- **Program execution:** load data and instruction to main memory
- **Access to I/O devices**: The OS provides a uniform interface that hides these details so programmers can access such devices using simple reads and writes.
- **Controlled access to files:** A system with multiple users, the OS may provide protection mechanisms

to control access to the files.

- **System access:** The access function must provide protection of resources and data from unauthorized users
- **Error detection and response:** OS must provide a response that clears the error condition with the least impact on running applications.
- **Accounting:** OS collect usage statistics for various resources and monitor performance parameters such as response time.
- **Instruction set architecture (ISA):** is the machine language instructions that a computer can follow and acts as boundary between hardware and software.
- **Application binary interface (ABI):** The ABI defines a standard for binary portability across programs.
- **Application programming interface (API):** The API gives a program access to the hardware resources and services available in a system through the user ISA supplemented with high-level language (HLL) library calls.

The Operating System as Resource Manager

- Computer's resources
  - I/O,
  - main and secondary memory,
  - and processor execution time.



Ease of Evolution of an Operating System

- A major OS will evolve over time for a number of reasons:
  - **Hardware upgrades plus new types of hardware:**
    - the use of graphics terminals and page-mode terminals instead of line-at-a-time scroll mode terminals affects OS design.
  - **New services:**
    - if it is found to be difficult to maintain good performance for users with existing tools, new measurement and control tools may be added to the OS.
  - **Fixes:**
    - Any OS has faults.

2.2 THE EVOLUTION OF OPERATING SYSTEMS

- Serial Processing
- Simple Batch Systems
- Multiprogrammed Batch Systems
- Time-Sharing Systems

Serial Processing

- The programmer interacted directly with the computer hardware; there was no OS.
- Programs in machine code were loaded via the input device (e.g., a card reader).
- If an error halted the program, the error condition was indicated by the lights. If the program proceeded to a normal completion, the output appeared on the printer.
- These early systems presented two main problems:
  - **Scheduling**
  - **Setup time**

Simple Batch Systems

- The wasted time due to scheduling and setup time was unacceptable.
- To improve utilization, the concept of a batch OS was developed.
- The central idea behind the simple batch-processing scheme is the use of a piece of software known as the **monitor** (type of OS).
- The user no longer has direct access to the processor.
- The user submits the job on cards or tape to a computer operator, who batches the jobs together sequentially and places the entire batch on an input device
- The monitor reads in jobs one at a time from the input device
- The current job is placed in the user program area, and control is passed to this job.
- When the job is completed, it returns control to the monitor, which immediately reads in the next job.
- The results of each job are sent to an output device, such as a printer, for delivery to the user.

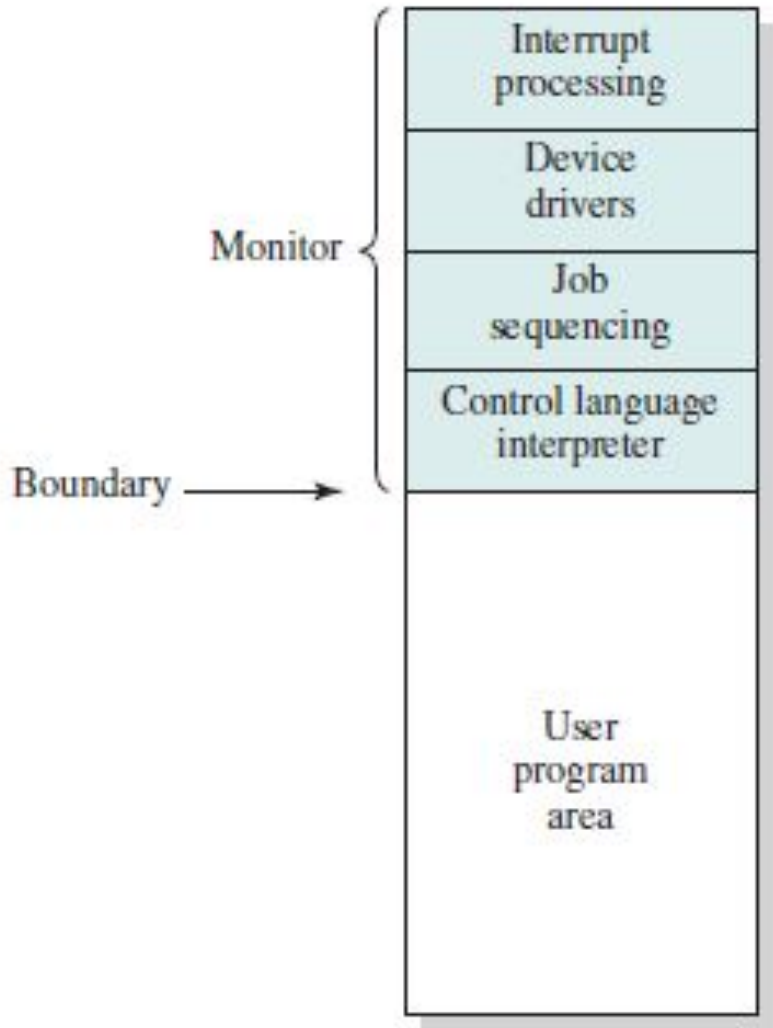


Figure 2.3 Memory Layout for a Resident Monitor

- Certain other hardware features are also desirable:

  - **Memory protection:** While the user program is executing, it must not alter the memory area containing the monitor

  - **Timer:** A timer is used to prevent a single job from monopolizing the system.

  - **Privileged instructions:** Certain machine level instructions are designated as privileged and can be executed only by the monitor.

  - **Interrupts**

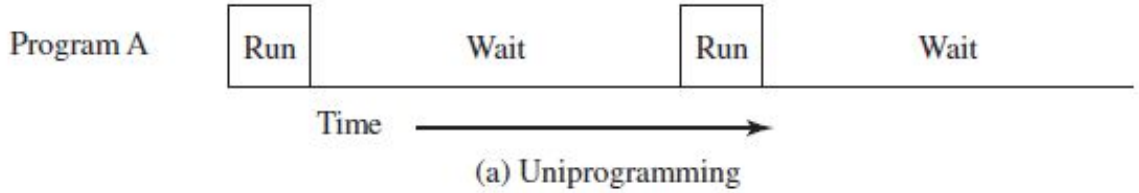
Multiprogrammed Batch Systems

- Even with the automatic job sequencing provided by a simple batch OS, the processor is often idle.

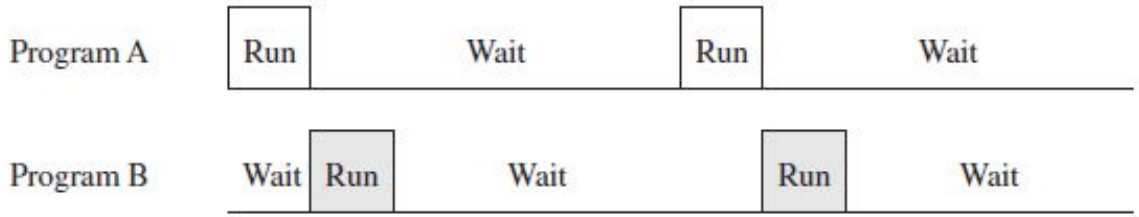
Read one record from file	15 μs
Execute 100 instructions	1 μs
Write one record to file	15 μs
Total	31 μs

$$\text{Percent CPU utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

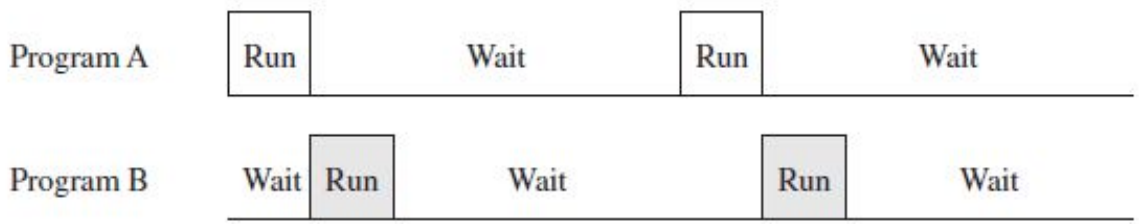
Figure 2.4 System Utilization Example



(a) Uniprogramming



(b) Multiprogramming with two programs



(c) Multiprogramming with three programs

Time-Sharing Systems

- multiprogramming allows the processor to handle multiple batch jobs at a time
- multiprogramming can also be used to handle multiple interactive jobs.
- processor time is shared among multiple users.
- In a time-sharing system, multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation.

Table 2.3 Batch Multiprogramming versus Time Sharing

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

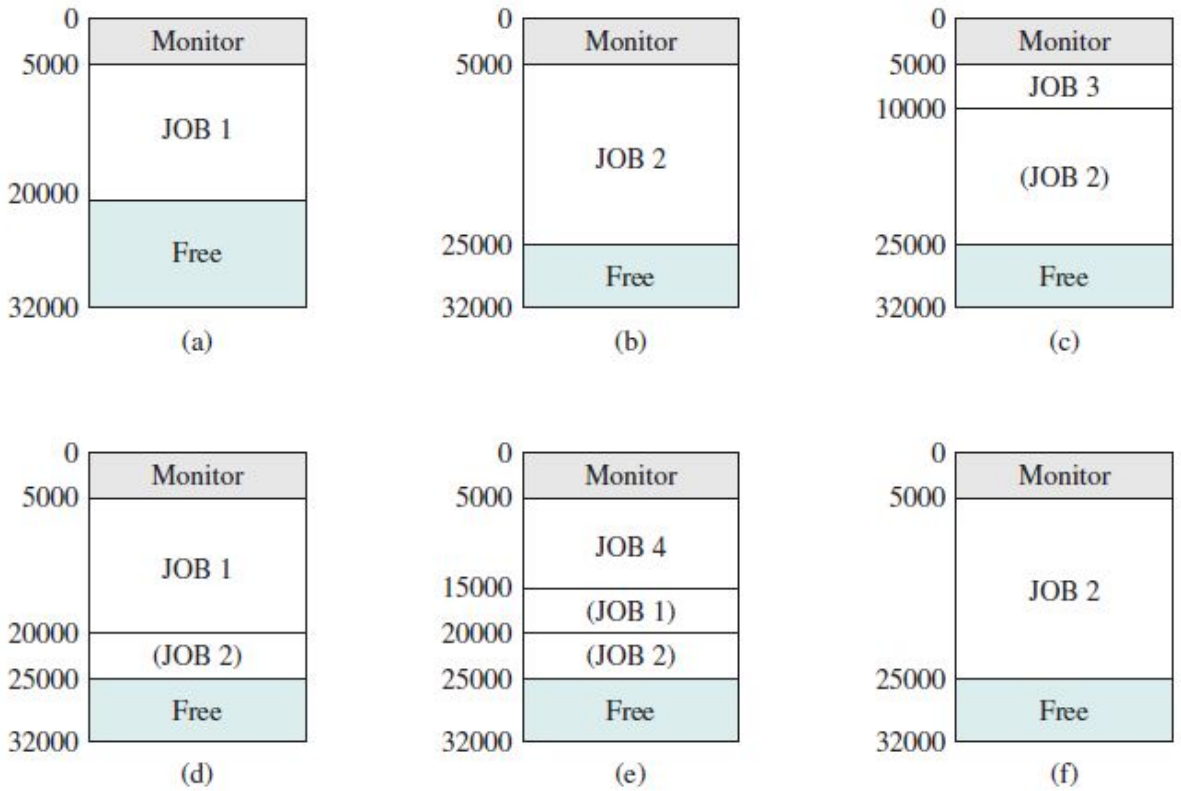


Figure 2.7 **CTSS Operation**

- JOB1: 15,000
- JOB2: 20,000
- JOB3: 5,000
- JOB4: 10,000

2.3 MAJOR ACHIEVEMENTS

- Four major theoretical advances in the development of operating systems:
  - Processes
  - Memory management
  - Information protection and security
  - Scheduling and resource management

The Process 1

- A program in execution.
- An instance of a program running on a computer.
- The entity that can be assigned to and executed on a processor.
- A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources.
- With many jobs in progress at any one time, it became impossible to analyze all of the possible combinations of sequences of events.

Process 2

- We can think of a process as consisting of three components:

1. An executable program

2. The associated data needed by the program (variables, work space, buffers, etc.)

3. The execution context / process state, of the program : The context includes all of the information the OS needs to manage the process, and the processor needs to execute the process properly.

- The context includes the contents of the various processor registers, such as the program counter and data registers. It also includes information of use to the OS, such as the priority of the process and whether the process is waiting for the completion of a particular I/O event.

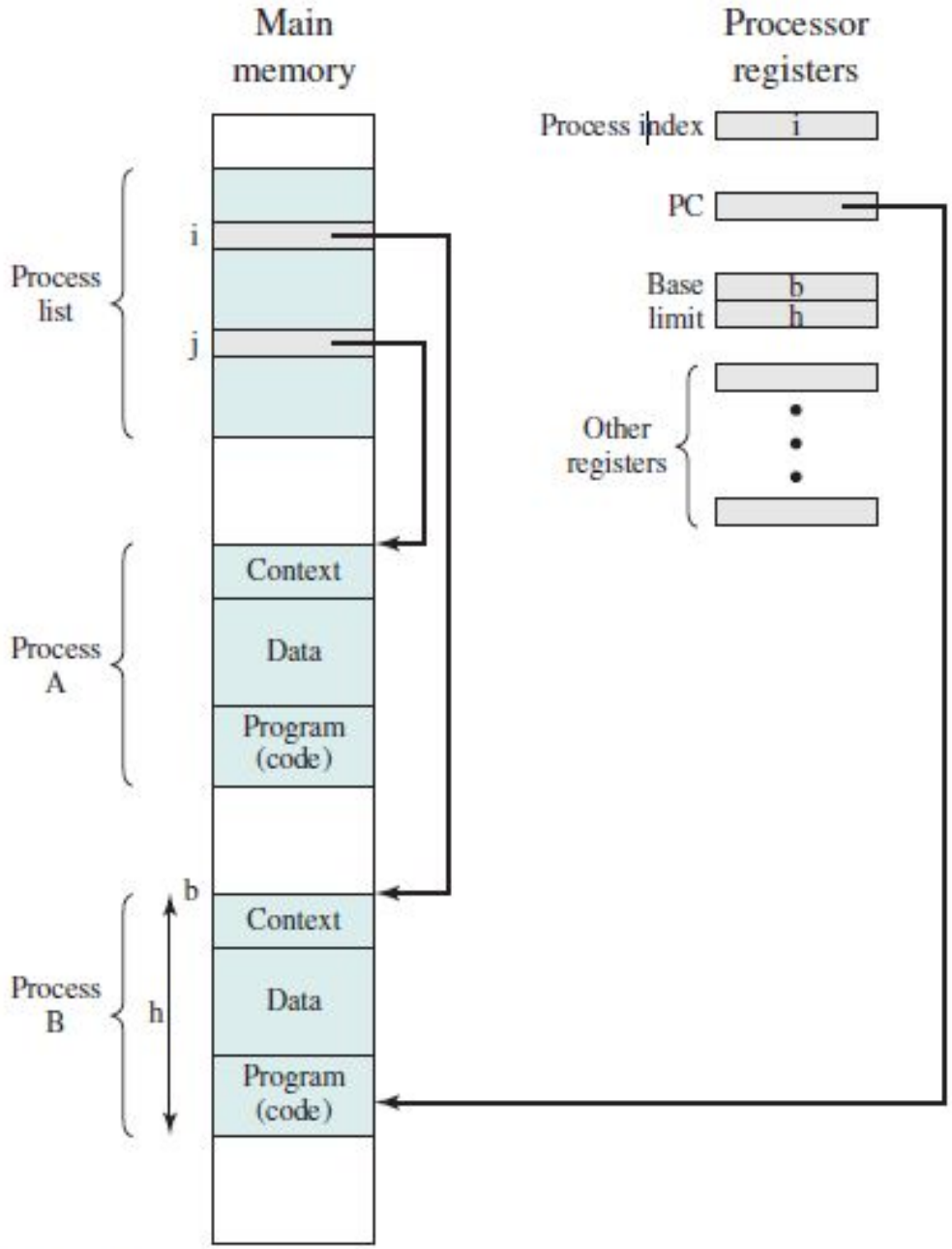


Figure 2.8 **Typical Process Implementation**

threads

- A single process, which is assigned certain resources, can be broken up into multiple, concurrent threads that execute cooperatively to perform the work of the process.
- This introduces a new level of parallel activity to be managed by the hardware and software.

Four main causes of Synchronization errors

- **Improper synchronization:** Improper design of the signaling mechanism can result in signals being lost or duplicate signals being received.
- **Failed mutual exclusion:** two users may attempt to edit the same file at the same time.
- **Nondeterminate program operation:** when programs share memory, and their execution is interleaved by the processor, they may interfere with each other by overwriting common memory areas in unpredictable ways.
- **Deadlocks:** Each process is waiting for the other program to release the desired resource.

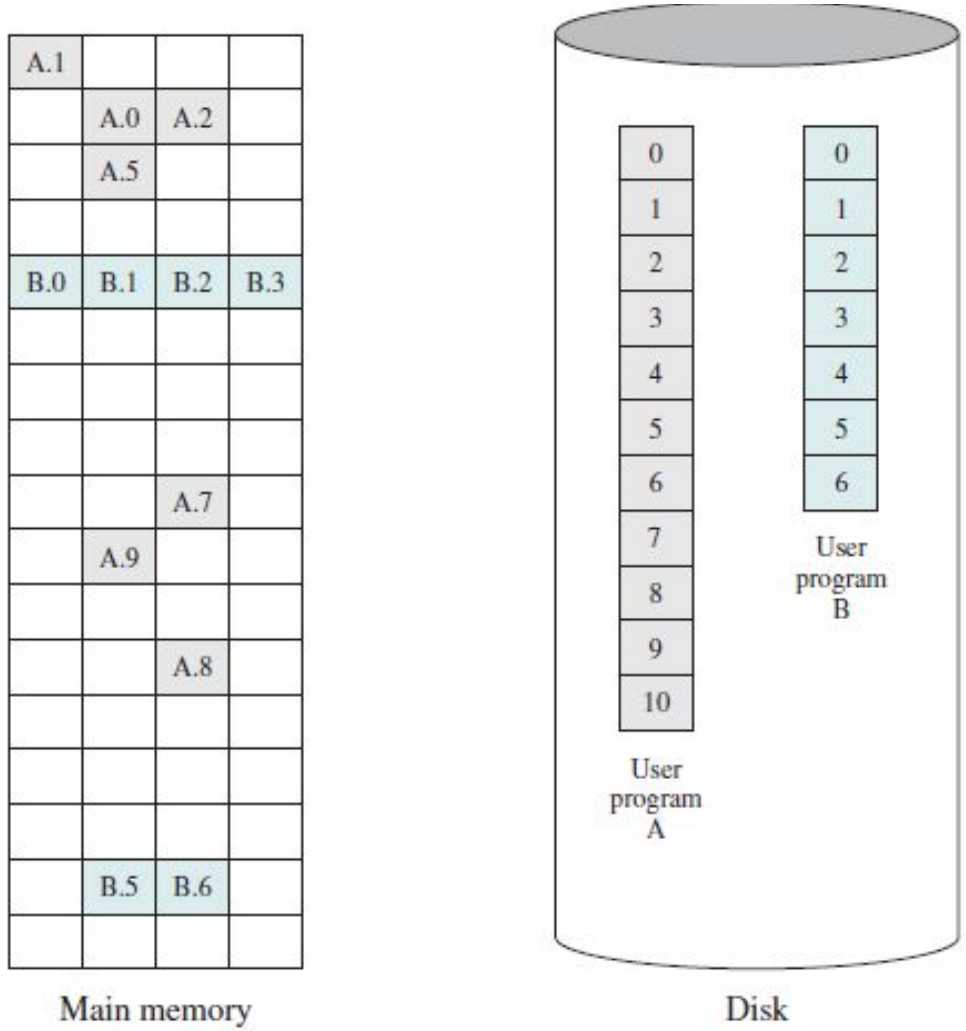
Memory Management

- **Process isolation:** The OS must prevent independent processes from interfering with each other's memory
- **Automatic allocation and management:** Programs should be dynamically allocated across the memory hierarchy as required.
- **Support of modular programming:** Programmers should be able to define program modules, and to dynamically create, destroy, and alter the size of modules.
- **Protection and access control:** The OS must allow portions of memory to be accessible in various ways by various users with out interfering with each other.
- **Long-term storage:** Many application programs require means for storing information for extended periods of time, after the computer has been powered down.

Virtual memory

- is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available.

- Virtual memory was conceived to meet the requirement of having multiple user jobs concurrently reside in main memory



Main memory consists of a number of fixed-length frames, each equal to the size of a page. For a program to execute, some or all of its pages must be in main memory.

Secondary memory (disk) can hold many fixed-length pages. A user program consists of some number of pages. Pages of all programs plus the OS are on disk, as are files.

Figure 2.9 Virtual Memory Concepts

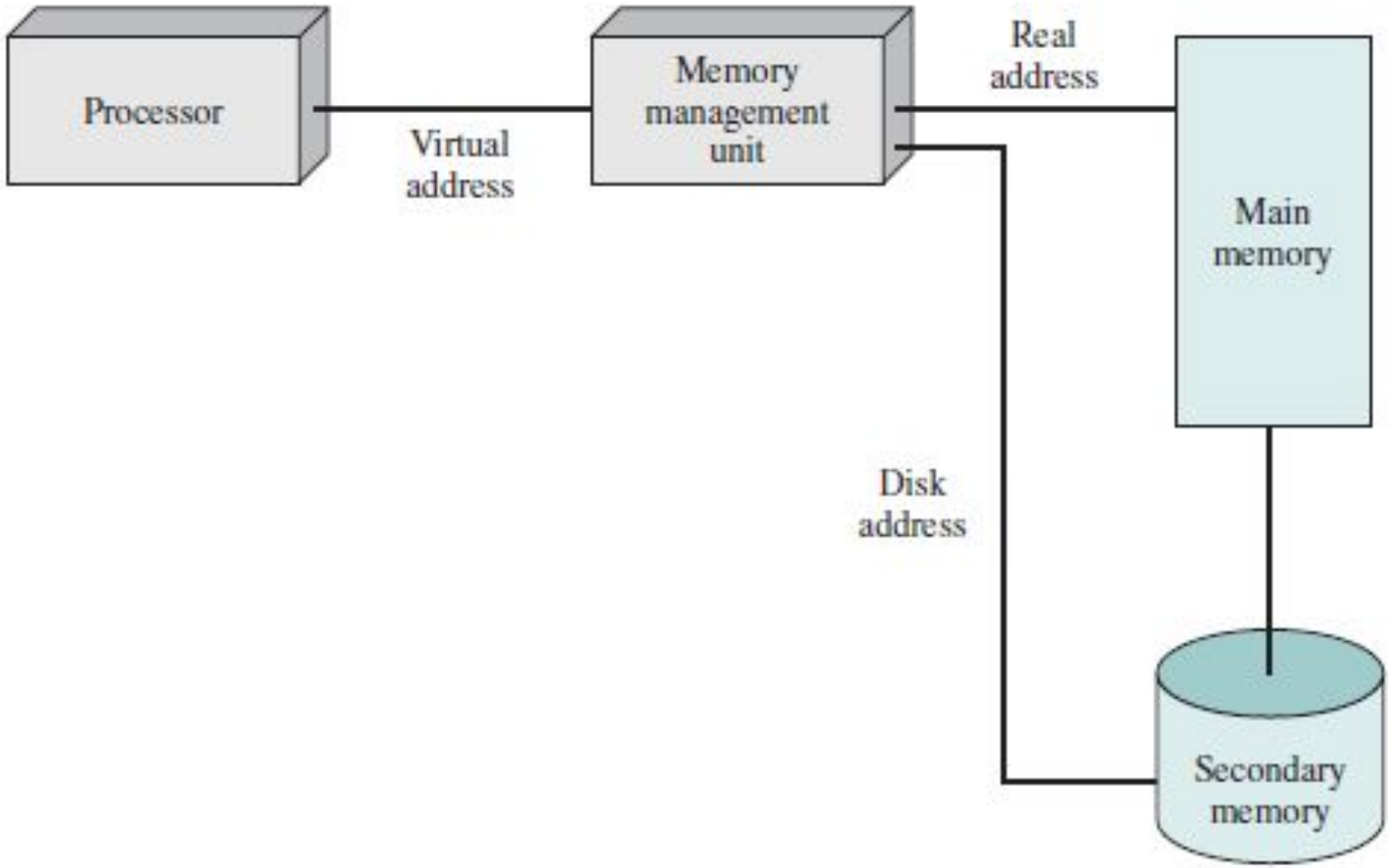


Figure 2.10 Virtual Memory Addressing

Information Protection and Security

- The growth in the use of time-sharing systems and, more recently, computer networks has brought with it a growth in concern for the protection of information.
- Security and protection as it relates to operating systems can be roughly grouped into four categories:
  - **Availability:** Concerned with protecting the system against interruption.
  - **Confidentiality:** Assures that users cannot read data for which access is unauthorized.
  - **Data integrity:** Protection of data from unauthorized modification.
  - **Authenticity:** Concerned with the proper verification of the identity of users and the validity of messages or data.

Scheduling and Resource Management

- **Fairness:** all processes that are competing for the use of a particular resource to be given approximately equal and fair access to that resource.
- **Differential responsiveness:** if a process is waiting for the use of an I/O device, the OS may wish to schedule that process for execution as soon as possible; the process can then immediately use the device, then release it for later demands from other processes.
- **Efficiency:** The OS should attempt to maximize throughput, minimize response time, and, in the case of time sharing, accommodate as many users as possible.

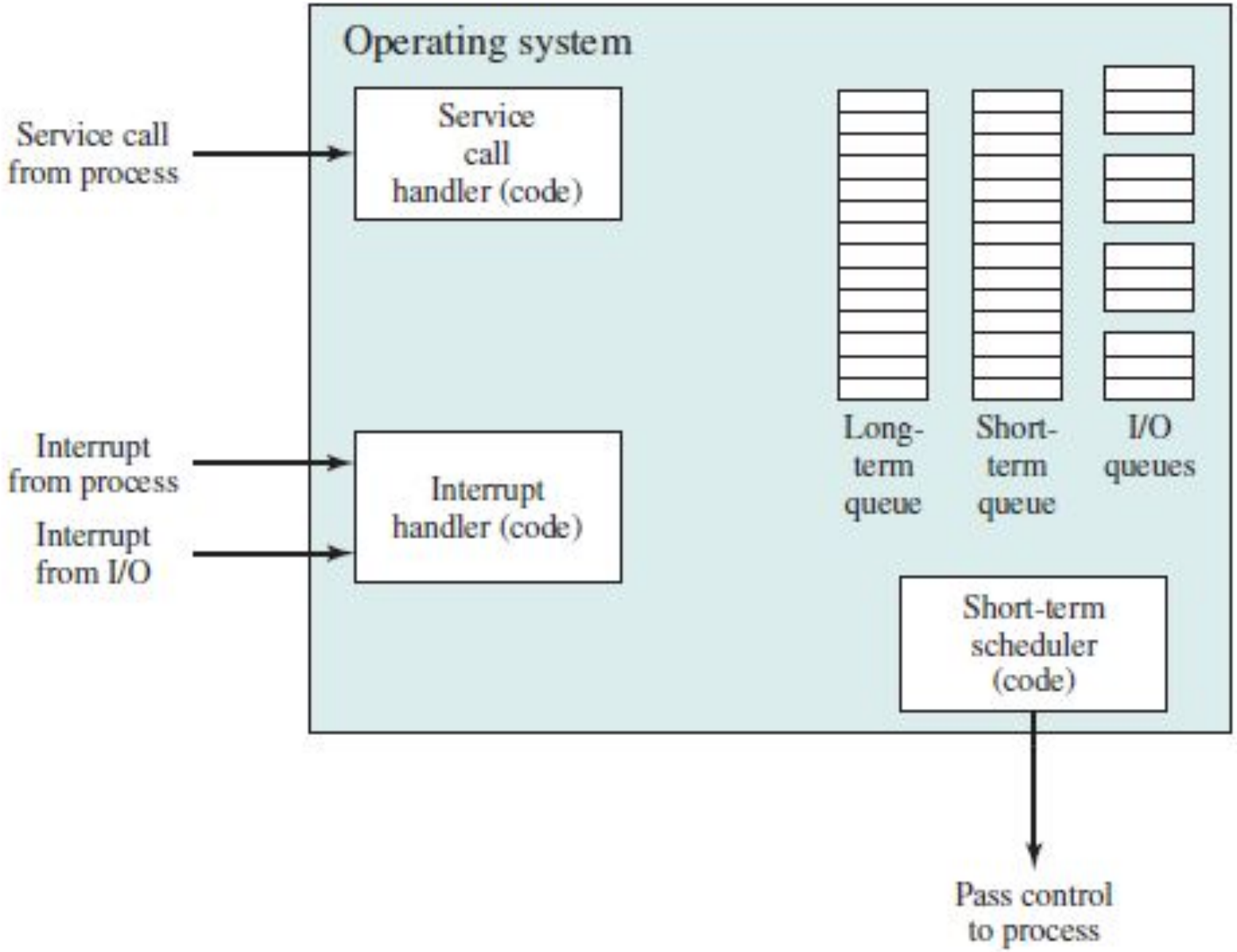


Figure 2.11 Key Elements of an Operating System for Multiprogramming

2.4 DEVELOPMENTS LEADING TO MODERN OPERATING SYSTEMS

- Until recently, most operating systems featured a large **monolithic kernel**. Most of what is thought of as OS functionality is provided in these large kernels, including scheduling, file system, networking, device drivers, memory management, and more.
- Typically, a monolithic kernel is implemented as a single process, with all elements sharing the same address space.
- A **microkernel** architecture assigns only a few essential functions to the kernel, including address space management, inter process communication (IPC), and basic scheduling. Other OS services are provided by processes, sometimes called servers, that run in user mode and are treated like any other application by the microkernel.